

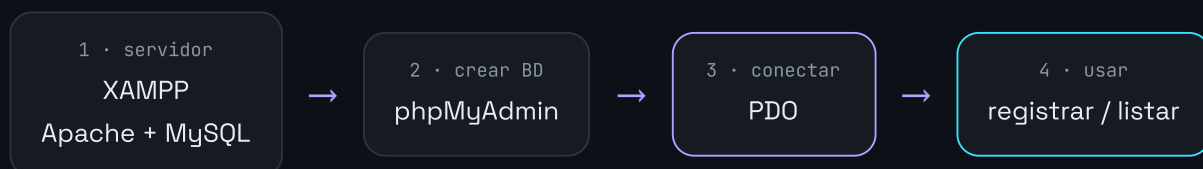


Conectar PHP con MySQL

Usando XAMPP y PDO — registro de usuarios de S.I.G.S.M.

Proyecto S.I.G.S.M. • 3.º Bachillerato Tecnológico • Polo Tecnológico Vista Linda

Vamos a trabajar sobre la tabla **usuario**, que es parte del núcleo de S.I.G.S.M. en todos los grupos. Cada usuario tiene un **rol**: **administrativo** o **medico**. El objetivo concreto de la práctica es resolver el **registro de usuarios**: guardarlos en la base y volver a leerlos, con PDO y sentencias preparadas.



CONTENIDO

1. Arrancar MySQL en XAMPP
2. Crear la tabla usuario
3. ¿Qué es PDO y por qué?
4. Conectar con PDO
5. Separar la conexión
6. Listar usuarios: SELECT
7. Consultas seguras (prepared)
8. Registrar un usuario: INSERT
9. Formulario de registro completo
10. Editar y dar de baja
11. Errores comunes
12. Checklist de la práctica

01 Arrancar MySQL en XAMPP

Además de **Apache** (que sirve tus `.php`), encendé **MySQL**, el motor de base de datos.

1. Abrí el **XAMPP Control Panel**.
2. Hací **Start** en **Apache** y en **MySQL** (ambas en verde).
3. Comprobá phpMyAdmin en `http://localhost/phpmyadmin`.

MySQL/MariaDB = el motor que guarda los datos. **phpMyAdmin** = una página web para administrarlo con el mouse (crear bases, tablas, ver registros).

02 Crear la tabla usuario

En phpMyAdmin, entrá a la pestaña **SQL**, pegá esto y ejecutá. Crea la base **sigism** y la tabla **usuario** con dos usuarios de ejemplo.

SQL · pestaña "SQL" de phpMyAdmin

```
CREATE DATABASE IF NOT EXISTS sigism CHARACTER SET utf8mb4;
USE sigism;

CREATE TABLE usuario (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  email       VARCHAR(120) NOT NULL UNIQUE,      -- no se puede repetir
  contrasenia VARCHAR(255) NOT NULL,             -- se guarda el HASH, nunca el texto
  rol         VARCHAR(20)  NOT NULL DEFAULT 'medico', -- 'administrativo' o 'medico'
  activo      TINYINT(1)   DEFAULT 1             -- 1 = activo, 0 = dado de baja
);

-- Usuarios de ejemplo. El hash corresponde a la clave 1234
INSERT INTO usuario (nombre, email, contrasenia, rol) VALUES
  ('Ana Torres', 'ana@sigism.uy', '$2y$10$thM8dWUpL04j059QdGn0zu9ub8ZvIPfUYrMTF5tQc7X0GbBHRPJG', 'medico'),
  ('Luis Pérez', 'luis@sigism.uy', '$2y$10$thM8dWUpL04j059QdGn0zu9ub8ZvIPfUYrMTF5tQc7X0GbBHRPJG', 'medico');
```

QUÉ HICIMOS

Una tabla con **clave primaria** autoincremental (**id**), el **email** como dato único, el **rol** con valor por defecto **medico**, y **activo** para la baja lógica. Los dos usuarios de ejemplo tienen la clave **1234** (guardada como hash).

POR QUÉ EL EMAIL ES UNIQUE

Sirve para que no existan dos usuarios con el mismo correo. Si alguien intenta registrarlo repetido, MySQL lo rechaza — y en el capítulo 9 vemos cómo avisarle al usuario con un mensaje claro.

03 ¿Qué es PDO y por qué lo usamos?

PDO (PHP Data Objects) es la herramienta de PHP para hablar con la base de datos. Es la que usamos en la cátedra por tres razones:

- > **Seguridad:** con **sentencias preparadas** evita la inyección SQL (capítulo 7).
- > **Portable:** el mismo código sirve para MySQL, PostgreSQL y otros motores.
- > **Ordenado:** maneja errores con excepciones y devuelve las filas como arrays asociativos.

REGLA DE LA CÁTEDRA

Siempre **PDO con sentencias preparadas**, y las contraseñas **siempre con hash** (nunca en texto plano).

04 Conectar con PDO

En XAMPP, los datos de conexión por defecto son:

DATO	VALOR EN XAMPP
Host (servidor)	localhost
Base de datos	sigsm
Usuario	root
Contraseña	"" (vacía)

conectar.php

```
<?php
    $host      = "localhost";
    $db        = "sigsm";
    $user      = "root";      // usuario por defecto de XAMPP
    $pass      = "";          // en XAMPP la clave de root está vacía
    $charset   = "utf8mb4";

    $dsn = "mysql:host=$host;dbname=$db;charset=$charset";

    try {
        $pdo = new PDO($dsn, $user, $pass, [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,    // errores como excepción
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC           // filas como array asociativo
        ]);

        echo "Conexión exitosa a la base sigsm";

    } catch (PDOException $e) {
```

```
    echo "Error de conexión: " . $e->getMessage();  
}
```

QUÉ SIGNIFICA CADA PARTE

- > `$dsn` — la "dirección" de la base: motor, host, nombre y charset.
- > `ERRMODE_EXCEPTION` — si algo falla, PHP **avisa** (no falla en silencio).
- > `FETCH_ASSOC` — cada fila llega como `["nombre" ⇒ "Ana", ...]`.
- > `try / catch` — si la conexión falla, entra al `catch` en vez de romper la página.

SOLO PARA DESARROLLO LOCAL

Usuario `root` sin contraseña sirve en XAMPP para aprender. En un servidor real se crea un usuario con permisos limitados y contraseña.

[↑ volver al índice](#)

05 Separar la conexión en su propio archivo

Poné la conexión **una sola vez** en un archivo aparte y traela donde la necesites.

conexion.php

```
<?php  
$dsn = "mysql:host=localhost;dbname=sigsm;charset=utf8mb4";  
  
try {  
    $pdo = new PDO($dsn, "root", "", [  
        PDO::ATTR_ERRMODE          ⇒ PDO::ERRMODE_EXCEPTION,  
        PDO::ATTR_DEFAULT_FETCH_MODE ⇒ PDO::FETCH_ASSOC  
    ]);  
} catch (PDOException $e) {  
    die("Error de conexión: " . $e->getMessage());  
}
```

usuarios.php · usa la conexión

```
<?php  
require "conexion.php"; // trae $pdo listo para usar  
  
// ... acá tus consultas usando $pdo
```

POR QUÉ CONVIENE

Si cambia la contraseña o el nombre de la base, lo tocás en **un solo lugar**. Es el primer paso hacia separar el proyecto en capas (presentación / negocio / datos).

06 Listar usuarios: SELECT

Para una consulta fija (sin datos del usuario) alcanza con `query()`.

usuarios.php

```
<?php
require "conexion.php";

$sql = "SELECT id, nombre, email, rol FROM usuario WHERE activo = 1";
$stmt = $pdo->query($sql);
$usuarios = $stmt->fetchAll();    // todas las filas en un array

foreach ($usuarios as $u) {
    echo $u["nombre"] . " (" . $u["rol"] . ") - " . $u["email"] . "<br>";
}
```

Mostrarlo en una tabla Bootstrap

```
<table class="table table-dark table-striped">
  <thead><tr><th>Nombre</th><th>Email</th><th>Rol</th></tr></thead>
  <tbody>
    <?php foreach ($usuarios as $u): ?>
      <tr>
        <td><?= $u["nombre"] ?></td>
        <td><?= $u["email"] ?></td>
        <td><?= $u["rol"] ?></td>
      </tr>
    <?php endforeach; ?>
  </tbody>
</table>
```

DOS ATAJOS ÚTILES

`<?= $var ?>` es lo mismo que `<?php echo $var; ?>`. Y `fetch()` trae una fila; `fetchAll()` trae **todas**.

↑ volver al índice

07 Consultas seguras: sentencias preparadas

Cuando la consulta usa **datos que vienen del usuario**, **nunca** los pegues directamente. Buscar un usuario por su email es el caso típico (lo vas a reusar en el login):

X PELIGROSO — no hagas esto

```
<?php
$email = $_POST["email"];
$sql = "SELECT * FROM usuario WHERE email = '$email'"; // ← inyección SQL
$stmt = $pdo->query($sql);
```

✓ SEGURO — sentencia preparada

```
<?php
require "conexion.php";

$email = $_POST["email"] ?? "";

$sql = "SELECT * FROM usuario WHERE email = :email";
$stmt = $pdo->prepare($sql);
$stmt->execute(["email" => $email]); // los datos van por separado
$usuario = $stmt->fetch();

if ($usuario) {
    echo "Usuario: " . $usuario["nombre"] . " . rol: " . $usuario["rol"];
} else {
    echo "No existe un usuario con ese email";
}
```

REGLA DE ORO

Todo dato que venga de afuera (`$_POST`, `$_GET`) va por `prepare()` + `execute()`. Sin excepciones.

08 Registrar un usuario: INSERT

Registrar un usuario es un INSERT preparado. La diferencia clave: la contraseña se guarda **hasheada** con `password_hash()`, nunca en texto.

registrar.php (fragmento)

```
<?php
require "conexion.php";
```

```

$nombre = $_POST["nombre"];
$email   = $_POST["email"];
$rol     = $_POST["rol"];

// La contraseña se convierte en hash antes de guardarla
$hash = password_hash($_POST["contrasenia"], PASSWORD_DEFAULT);

$sql = "INSERT INTO usuario (nombre, email, contrasenia, rol)
        VALUES (:nombre, :email, :contrasenia, :rol)";

$stmt = $pdo->prepare($sql);
$stmt->execute([
    "nombre"      => $nombre,
    "email"       => $email,
    "contrasenia" => $hash,
    "rol"         => $rol
]);

echo "Usuario registrado con id " . $pdo->lastInsertId();

```

NUNCA GUARDES LA CONTRASEÑA EN TEXTO

`password_hash()` la transforma en un texto irreversible (bcrypt). Para el login, más adelante, se compara con `password_verify()`. Así, aunque alguien vea la base, no puede leer las claves.

09 Formulario de registro completo

Este es el archivo que resuelve la práctica: muestra el formulario, valida con el **patrón guardián**, hashlea la contraseña y guarda. Si el email ya existe, avisa sin romperse.

registrar.php

```

<?php
require "conexion.php";
$mensaje = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {

    $nombre      = trim($_POST["nombre"] ?? "");
    $email       = trim($_POST["email"] ?? "");
    $contrasenia = $_POST["contrasenia"] ?? "";
    $rol         = $_POST["rol"] ?? "medico";

    // Patrón guardián: cortamos ante cada dato inválido
    if ($nombre === "") $mensaje = "Falta el nombre";
    elseif ($email === "") $mensaje = "Falta el email";

```

```

elseif ($contrasenia === "") $mensaje = "Falta la contraseña";
elseif ($rol === "administrativo" && $rol === "medico")
    $mensaje = "Rol inválido";

else {
    // Todo válido: guardamos
    try {
        $hash = password_hash($contrasenia, PASSWORD_DEFAULT);

        $sql = "INSERT INTO usuario (nombre, email, contrasenia, rol)
                VALUES (:nombre, :email, :contrasenia, :rol)";
        $stmt = $pdo->prepare($sql);
        $stmt->execute([
            "nombre"      => $nombre,
            "email"       => $email,
            "contrasenia" => $hash,
            "rol"         => $rol
        ]);

        $mensaje = "Usuario registrado correctamente";

    } catch (PDOException $e) {
        // 23000 = viola una restricción (email repetido)
        if ($e->getCode() === "23000") {
            $mensaje = "Ese email ya está registrado";
        } else {
            $mensaje = "Error al registrar";
        }
    }
}

?>
<!DOCTYPE html>
<html lang="es" data-bs-theme="dark">
<head>
    <meta charset="utf-8">
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="st
</head>
<body class="container py-4" style="max-width:520px;">

    <h1 class="h4 mb-3">Registrar usuario</h1>

    <?php if ($mensaje === ""): ?>
        <div class="alert alert-info"><?= $mensaje ?></div>
    <?php endif; ?>

    <form method="POST">
        <input class="form-control mb-2" name="nombre" placeholder="Nombre">
        <input class="form-control mb-2" name="email" type="email" placeholder="Email">
        <input class="form-control mb-2" name="contrasenia" type="password" placeholder="Contraseñ
        <select class="form-select mb-3" name="rol">
            <option value="medico">Médico</option>
            <option value="administrativo">Administrativo</option>
        </select>

```



```
<button class="btn btn-primary">Registrar</button>
</form>

</body>
</html>
```

PARA LA PRÁCTICA

Con este archivo y la tabla `usuario`, cada estudiante puede registrar usuarios reales y verlos aparecer en el listado (capítulo 6) y en phpMyAdmin. Es la base del módulo de login que viene después.

[↑ volver al índice](#)

10 Editar y dar de baja

Actualizar el rol (UPDATE)

```
<?php
$sql = "UPDATE usuario SET rol = :rol WHERE id = :id";
$stmt = $pdo->prepare($sql);
$stmt->execute([
    "rol" => $_POST["rol"],
    "id"  => $_POST["id"]
]);
```

Baja lógica (convención de la cátedra)

No borramos usuarios con `DELETE`: los marcamos inactivos. Así se conserva el historial.

```
<?php
// En lugar de DELETE, marcamos activo = 0
$sql = "UPDATE usuario SET activo = 0 WHERE id = :id";
$stmt = $pdo->prepare($sql);
$stmt->execute(["id" => $_POST["id"]]);
```

POR ESO EL LISTADO LLEVA WHERE ACTIVO = 1

Los usuarios dados de baja siguen en la tabla, pero no aparecen en el listado ni pueden iniciar sesión.

11 Errores comunes

Mensaje / Síntoma	Causa y Solución
MySQL no arranca (rojo en XAMPP)	Puerto <code>3306</code> ocupado por otro MySQL como servicio de Windows. Detené ese servicio o cambiá el puerto en XAMPP.
<code>[1045] Access denied</code>	Usuario o contraseña incorrectos. En XAMPP es <code>root</code> con clave vacía .
<code>[1049] Unknown database</code>	La base no existe o el nombre no coincide. Revisá que creaste <code>sigsm</code> .
<code>[23000] Duplicate entry</code>	Intentaste registrar un email repetido . Es lo que capturamos en el capítulo 9.
Tildes o ñ como <code>?</code>	Falta <code>charset=utf8mb4</code> en el DSN, o la tabla no está en utf8mb4.
Página en blanco	Activá <code>display_errors</code> (ver mini-manual de XAMPP) para ver la excepción real.

12 Checklist de la práctica

- > Apache y MySQL en verde en XAMPP.
- > La base `sigsm` y la tabla `usuario` existen en phpMyAdmin.
- > La conexión está en `conexion.php` y se incluye con `require`.
- > El registro guarda la contraseña con `password_hash()` (nunca en texto).
- > Todas las consultas con datos del usuario usan `prepare()` + `execute()`.
- > El listado filtra por `activo = 1`; la baja es lógica (`activo = 0`).
- > El rol solo puede ser `administrativo` o `medico`.
- > Puedo explicar oralmente qué hace cada línea, sin ayuda de IA.

USO DE IA

Podés apoyarte en IA para entender, pero todo lo que entregues lo tenés que poder **defender oralmente sin ayuda**. Si no podés explicar por qué escribiste una línea, esa línea no cuenta.

